

BCA — Feature Engineering

Handling Missing Data & Data Cleaning

Lecture Notes with Real Datasets — Titanic & House Prices

Topics Covered

Types of Missing Data (MCAR/MAR/MNAR) | Detection | Drop / Impute / KNN | Duplicates |
Outliers (IQR) | Inconsistent Formats | Complete Pipeline

PART 1: Handling Missing Data

1.1 What is Missing Data?

Missing data means that for some rows in your dataset, one or more column values are absent. In Python and pandas, missing numerical values appear as NaN (Not a Number) and missing text values appear as None.

Classroom Story: Imagine you are running a survey in your college. You ask 500 students about their age, CGPA, hometown, and monthly expenses. Some students skip the age question. Some skip CGPA. Some leave hometown blank. When you open your Excel file — those cells are empty. That is missing data.

Why Does It Matter?

Machine learning algorithms cannot process NaN values. If you pass a DataFrame with missing values into sklearn, it will throw an error. Beyond errors, missing data can introduce bias — if the pattern of missingness is not random, your model can learn incorrect patterns.

1.2 The Three Types of Missing Data

Before deciding how to handle missing data, you must understand WHY it is missing. The reason determines the correct strategy.

MCAR — Missing Completely At Random

Definition: The probability of a value being missing has absolutely no relationship with any other variable in the dataset — including the variable itself.

Example: A lab technician accidentally spills coffee on 15 random survey forms. Those 15 forms are now incomplete — purely by accident. Who got coffee spilled on their form has nothing to do with their age, income, or any other characteristic.

Strategy: Safe to drop rows or apply simple imputation. No bias is introduced.

MAR — Missing At Random

Definition: The probability of a value being missing depends on OTHER observed variables — not on the missing variable itself.

Example (Titanic Dataset): Third class passengers are more likely to have their age missing than first class passengers. The missingness of age depends on pclass — not on the age value itself.

Strategy: Impute carefully using related columns (group-wise imputation). Simple mean/median may introduce bias.

MNAR — Missing Not At Random

Definition: The probability of a value being missing depends on the value itself — the very value that is missing.

Example: In an income survey, very high earners tend to leave their income field blank because they do not want to disclose it. The income value being missing is directly caused by the income being high.

Strategy: Most dangerous. Dropping rows introduces serious bias. Must be flagged, modelled, or handled with domain knowledge. Cannot be fixed by simple imputation.

1.3 Real Dataset — Titanic

The Titanic dataset is the perfect teaching tool for missing data. It is freely available in seaborn, every student has heard of the Titanic, and it has three distinctly different types of missingness.

Step 1 — Load and Inspect

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load Titanic — built into seaborn, no download needed
df = sns.load_dataset('titanic')

print('Shape:', df.shape)
print(df.head())
```

```
Shape: (891, 15)

   survived  pclass    sex  age  sibsp  parch    fare embarked
0         0      3  male  22.0     1     0    7.2500         S
1         1      1  female  38.0     1     0   71.2833         C
2         1      3  female  26.0     0     0    7.9250         S
3         1      1  female  35.0     1     0   53.1000         S
4         0      3   male  35.0     0     0    8.0500         S
```

Step 2 — Detect Missing Values

```
# Count and percentage of missing values per column
missing_count = df.isnull().sum()
missing_percent = (df.isnull().sum() / len(df)) * 100

missing_summary = pd.DataFrame({
    'Missing Count': missing_count,
    'Missing Percent': missing_percent.round(2)
})
```

```

}))

# Show only columns with missing values
print(missing_summary[missing_summary['Missing Count'] > 0]
      .sort_values('Missing Percent', ascending=False))

```

```

      Missing Count  Missing Percent
deck              688             77.28  <- drop this column
age              177             19.87  <- impute carefully
embarked          2              0.22  <- mode imputation

```

```

# Visual heatmap of missing values
plt.figure(figsize=(12, 5))
sns.heatmap(df.isnull(), yticklabels=False, cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap – Yellow = Missing')
plt.tight_layout()
plt.show()
# Yellow streaks show where data is missing
# deck column is almost entirely yellow = 77% missing

```

Step 3 — Identify the Type of Missingness

```

# Is age missingness related to survival? (checking for MAR)
print('Survival rate – age present vs age missing:')
print(df.groupby(df['age'].isnull())['survived'].mean())

# Is age missingness related to passenger class?
print('\nMissing age count by passenger class:')
print(df.groupby('pclass')['age'].apply(lambda x: x.isnull().sum()))

```

```

Survival rate – age present vs age missing:
age
False    0.4061  <- age present: 40.6% survived
True     0.2938  <- age missing: 29.4% survived

Missing age count by passenger class:
pclass
1         30
2         11
3        136  <- 3rd class has most missing ages (MAR!)

```

Teaching Insight:

The survival rate is different for rows where age is missing vs present. This tells us the missingness is NOT completely random (not MCAR). And since most missing ages belong to 3rd class passengers, the missingness depends on pclass — a separate observed variable. This is MAR. The correct fix is group-wise imputation, not a single overall median.

Step 4 — Apply the Correct Strategy to Each Column

Different columns need different strategies — based on the percentage missing and the type of missingness.

Strategy A: Drop the deck column (77% missing)

```
print('Before drop:', df.shape)
df_cleaned = df.drop(columns=['deck'])
print('After drop: ', df_cleaned.shape)
# Output:
# Before drop: (891, 15)
# After drop:  (891, 14)  <- one column removed
```

Rule of Thumb

If more than 40-50% of a column's values are missing, it is usually better to drop the column entirely. A column that is mostly missing provides very little useful information to the model.

Strategy B: Mode imputation for embarked (2 missing)

```
print('Embarked value counts:')
print(df_cleaned['embarked'].value_counts())

# Fill with the most common port (mode)
df_cleaned['embarked'] = df_cleaned['embarked'].fillna(
    df_cleaned['embarked'].mode()[0]
)

print('Missing after fix:', df_cleaned['embarked'].isnull().sum())
# Output:
# S      644    <- most common
# C      168
# Q       77
# Missing after fix: 0
```

Strategy C: Group-wise Median Imputation for age (19.87% missing)

```
# Check median age by pclass and sex
print(df_cleaned.groupby(['pclass', 'sex'])['age'].median())

# Output:
# pclass  sex
# 1      female    35.0
# 1      male      40.0
# 2      female    28.0
# 2      male      30.0
```

```
# 3      female      21.5
# 3      male        25.0

# Fill missing age using group median
def fill_age(row):
    if pd.isnull(row['age']):
        return df_cleaned.groupby(
            ['pclass', 'sex'])['age'].median()[row['pclass'], row['sex']]
    return row['age']

df_cleaned['age'] = df_cleaned.apply(fill_age, axis=1)
print('Missing age after imputation:', df_cleaned['age'].isnull().sum())
# Output: Missing age after imputation: 0
```

Why group-wise and not overall median?

The overall median age is 28. But a 1st class male passenger and a 3rd class female passenger have very different typical ages (40 vs 21.5). Filling both with 28 introduces noise. Group-wise imputation is more realistic and better preserves the natural patterns in the data.

Strategy D: KNN Imputation (Advanced — for completeness)

```
from sklearn.impute import KNNImputer

# Select numerical columns only for KNN
df_knn = df[['pclass', 'age', 'sibsp', 'parch', 'fare']].copy()

# KNN finds the 5 most similar passengers and uses their average age
knn_imputer = KNNImputer(n_neighbors=5)
df_knn_imputed = pd.DataFrame(
    knn_imputer.fit_transform(df_knn),
    columns=df_knn.columns
)

print('Missing after KNN:', df_knn_imputed['age'].isnull().sum())
# Output: Missing after KNN: 0

# KNN imputes based on similar passengers:
# Row 5 (age missing) -> gets 29.3 because 5 similar passengers
#                        had ages around 29
# Row 17 (age missing, child profile) -> gets 3.1
```

1.4 Missing Data Strategies — Summary Table

Strategy	When to Use	Code	Risk
Drop rows	Less than 5% missing, MCAR	<code>df.dropna()</code>	Data loss

Drop column	More than 40-50% missing	<code>df.drop(columns=['col'])</code>	Lose feature entirely
Mean imputation	Numerical, normal distribution, no outliers	<code>fillna(df[col].mean())</code>	Reduces variance
Median imputation	Numerical, skewed data or outliers present	<code>fillna(df[col].median())</code>	Slightly better than mean
Mode imputation	Categorical columns	<code>fillna(df[col].mode()[0])</code>	Ignores relationships
Group-wise imputation	MAR — missingness related to other columns	<code>groupby().median()</code>	More realistic than simple
KNN Imputation	When data has strong patterns between features	<code>KNNImputer(n_neighbors=5)</code>	Computationally expensive

PART 2: Data Cleaning

2.1 What is Data Cleaning?

Data cleaning is the process of identifying and fixing errors, inconsistencies, and noise in a dataset before it is used for analysis or modelling. Missing data is one part of data cleaning — but there are several other problems that appear in real-world data.

Classroom Story: You receive a register with student marks. Some entries say 'PASS', some say 'pass', some say 'P'. Some marks are 850 out of 100 — clearly a typing error. One student's name appears twice with slightly different details — a duplicate. The date column has entries like '01-01-2024', '1 Jan 2024', and '2024/01/01' — all the same date, three different formats. Cleaning all of this before analysis is Data Cleaning.

2.2 Real Dataset — House Prices (Dirty Version)

We will use a realistic house prices dataset that has been intentionally made dirty — with the same types of problems that appear in actual client data.

```
import pandas as pd
import numpy as np

data = {
    'id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'area_sqft': [1200, 1500, 980, 2200, 1800, 1500,
                  9999999, 1100, 2500, 1650],
    'bedrooms': [2, 3, 2, 4, 3, 3, 2, 2, 4, 3],
    'price': [4500000, 6200000, 3800000, 8500000,
              7200000, 6200000, 450000,
              4100000, 9800000, 6100000],
    'location': ['Delhi', 'mumbai', 'DELHI', 'Bangalore',
                 'Mumbai', 'MUMBAI', 'Delhi',
                 'bangalore', 'Delhi', 'Bangalore'],
    'age_years': [5, 10, 3, 15, 8, 10, 6, 4, 12, 7],
    'date_listed': ['2024-01-15', '15/02/2024', '2024-03-20',
                    'March 5 2024', '2024-01-30', '2024-02-15',
                    '2024-01-10', '2024-04-01',
                    '2024-02-28', '2024-03-15'],
    'garage': ['Yes', 'No', 'yes', 'YES', 'no',
               'No', 'Yes', 'N', 'Y', 'yes']
}

df = pd.DataFrame(data)
print(df)
```

Ask Students:

Before running any code — look at this dataset and spot as many problems as you can.
Location column: 'Delhi', 'DELHI', 'delhi' — same city, different formats. Area: 9,999,999 sq ft — clearly impossible. Garage: 'Yes', 'yes', 'YES', 'Y' — four ways to say the same thing.
Date: three different formats in one column. Rows 2 and 6 (id=2 and id=6): same area, price, and location — duplicate!

2.3 Problem 1 — Duplicate Rows

Why duplicates happen:

- Form submitted twice by the same user
- Data merged from two sources without deduplication
- System error that recorded the same transaction twice

```
# Check for exact duplicate rows
print('Total rows:', len(df))
print('Duplicate rows:', df.duplicated().sum())

# Check duplicates on specific meaningful columns
print('\nDuplicate based on area + price + location:')
dup_mask = df.duplicated(
    subset=['area_sqft', 'price', 'location'], keep=False
)
print(df[dup_mask][['id', 'area_sqft', 'price', 'location']])
```

```
Total rows: 10
Duplicate rows: 0    <- exact duplicates: 0

Duplicate based on area + price + location:
   id  area_sqft  price location
1   2         1500  6200000  mumbai
5   6         1500  6200000  MUMBAI    <- same property!
```

```
# Fix: standardize location first, then drop duplicates
df['location'] = df['location'].str.strip().str.title()

before = len(df)
df = df.drop_duplicates(subset=['area_sqft', 'price', 'location'])
print(f'Rows removed: {before - len(df)}')
print(f'Rows remaining: {len(df)}')
# Output:
# Rows removed: 1
# Rows remaining: 9
```

2.4 Problem 2 — Outliers

An outlier is a value that is extremely far from the rest of the data. Outliers can be genuine (a real extreme case) or errors (wrong data entry). You must distinguish between the two.

IQR Method — The Standard Way to Detect Outliers

$$\begin{aligned} \text{IQR} &= Q3 - Q1 \\ \text{Lower Bound} &= Q1 - 1.5 * \text{IQR} \\ \text{Upper Bound} &= Q3 + 1.5 * \text{IQR} \end{aligned}$$

Analogy: IQR captures the middle 50% of your data. Anything more than 1.5 times the IQR beyond either edge of this middle 50% is considered an outlier. It is like saying — if a student's marks are more than 1.5 standard ranges beyond the typical spread, something unusual happened.

```
# Check area_sqft statistics
print(df['area_sqft'].describe())
```

```
count      10.000000
mean     1019077.900000  <- mean is destroyed by outlier
std      3159832.147000
min       980.000000
25%      1200.000000
50%      1575.000000
75%      2125.000000
max     9999999.000000  <- clearly a data entry error
```

```
# Apply IQR method
Q1 = df['area_sqft'].quantile(0.25)
Q3 = df['area_sqft'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print(f'Q1: {Q1} | Q3: {Q3} | IQR: {IQR}')
print(f'Lower Bound: {lower:.0f}')
print(f'Upper Bound: {upper:.0f}')

# Find outlier rows
outliers = df[(df['area_sqft'] < lower) | (df['area_sqft'] > upper)]
print('\nOutlier rows:')
print(outliers[['id', 'area_sqft', 'price']])

# Remove outliers
df = df[(df['area_sqft'] >= lower) & (df['area_sqft'] <= upper)]
print(f'\nRows after outlier removal: {len(df)}')
```

```

Q1: 1200.0 | Q3: 2125.0 | IQR: 925.0
Lower Bound: -188
Upper Bound: 3512

Outlier rows:
  id area_sqft price
6  7   9999999 450000  <- area 9.9M sqft is impossible

Rows after outlier removal: 8

```

Drop vs Cap (Winsorize):

When an outlier is clearly a data entry error (like 9,999,999 sq ft), drop the row. When an outlier is a genuine extreme value that you want to keep but reduce its influence, replace it with the boundary value (upper or lower bound). This is called Winsorizing or capping.

```

# Winsorize instead of dropping (cap at boundary)
df['area_sqft'] = df['area_sqft'].clip(lower=lower, upper=upper)
# This keeps the row but replaces 9999999 with 3512

```

2.5 Problem 3 — Inconsistent Text Formatting

When text data has multiple representations of the same value — uppercase, lowercase, abbreviations — it gets treated as different categories by the model. This is one of the most common and most damaging problems in real datasets.

```

# Location column - multiple formats for same city
print('Location values BEFORE:')
print(df['location'].unique())
# ['Delhi' 'mumbai' 'DELHI' 'Bangalore' 'Mumbai' 'MUMBAI' 'bangalore']

# Fix: strip whitespace, convert to Title Case
df['location'] = df['location'].str.strip().str.title()
print('\nLocation values AFTER:')
print(df['location'].unique())
# ['Delhi' 'Mumbai' 'Bangalore'] <- only 3 clean values

# Garage column - Yes/No/Y/N/yes/YES
print('\nGarage values BEFORE:', df['garage'].unique())

df['garage'] = df['garage'].str.strip().str.upper()
df['garage'] = df['garage'].map({
    'YES': 'Yes', 'Y': 'Yes',
    'NO': 'No', 'N': 'No'
})
print('Garage values AFTER:', df['garage'].unique())
# ['Yes' 'No']

```

2.6 Problem 4 — Inconsistent Date Formats

```
print('Date formats BEFORE:')
print(df['date_listed'].unique())
# ['2024-01-15'    <- ISO format
#  '15/02/2024'    <- DD/MM/YYYY
#  'March 5 2024'  <- written out in words
#  '2024-03-20']   <- ISO format

# pd.to_datetime handles mixed formats automatically
df['date_listed'] = pd.to_datetime(
    df['date_listed'],
    dayfirst=True,          # for DD/MM/YYYY format
    infer_datetime_format=True
)

print('\nDtype after fix:', df['date_listed'].dtype)
print(df['date_listed'])
# Dtype: datetime64[ns]    <- proper datetime now
# All dates in consistent YYYY-MM-DD format

# Now you can extract useful features from date
df['month_listed'] = df['date_listed'].dt.month
df['year_listed']  = df['date_listed'].dt.year
```

2.7 Problem 5 — Wrong Data Types

```
# Always check dtypes first thing in any project
print(df.dtypes)
# id                int64
# area_sqft         int64
# bedrooms           int64
# price             int64
# location          object    <- should be 'category'
# age_years         int64
# date_listed       object    <- should be datetime
# garage            object    <- should be 'category'

# Fix data types
df['date_listed'] = pd.to_datetime(df['date_listed'])
df['garage']      = df['garage'].astype('category')
df['location']    = df['location'].astype('category')

print('\nAfter fixing:')
print(df.dtypes)
# date_listed       datetime64[ns]
# garage            category
# location           category
```

2.8 Complete Cleaning Pipeline

This is the most important code block in this section. Show students how all the individual cleaning steps are combined into one reusable function. This is exactly how data cleaning is done in real projects.

```
def clean_house_data(df):  
    """  
    Complete data cleaning pipeline for house prices dataset.  
    Always apply steps in this order – sequence matters.  
    """  
    df = df.copy()    # never modify original data  
  
    print(f'Step 0 – Original shape: {df.shape}')  
  
    # Step 1 – Fix inconsistent text formatting  
    df['location'] = df['location'].str.strip().str.title()  
    df['garage'] = df['garage'].str.strip().str.upper()  
    df['garage'] = df['garage'].map({'YES': 'Yes', 'Y': 'Yes',  
                                   'NO': 'No', 'N': 'No'})  
    print('Step 1 – Text formatting fixed')  
  
    # Step 2 – Fix date formats  
    df['date_listed'] = pd.to_datetime(  
        df['date_listed'], dayfirst=True, infer_datetime_format=True  
    )  
    print('Step 2 – Date formats fixed')  
  
    # Step 3 – Remove duplicates  
    before = len(df)  
    df = df.drop_duplicates(subset=['area_sqft', 'price', 'location'])  
    print(f'Step 3 – Duplicates removed: {before-len(df)} rows')  
  
    # Step 4 – Remove outliers using IQR method  
    for col in ['area_sqft', 'price']:  
        Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)  
        IQR = Q3 - Q1  
        before = len(df)  
        df = df[  
            (df[col] >= Q1 - 1.5*IQR) &  
            (df[col] <= Q3 + 1.5*IQR)  
        ]  
        print(f'Step 4 – Outliers in {col}: {before-len(df)} removed')  
  
    # Step 5 – Fix data types  
    df['garage'] = df['garage'].astype('category')  
    df['location'] = df['location'].astype('category')  
    print('Step 5 – Data types fixed')  
  
    print(f'\nFinal shape: {df.shape}')  
    print('Cleaning complete!')  
    return df  
  
df_clean = clean_house_data(df.copy())
```

```

Step 0 - Original shape: (10, 8)
Step 1 - Text formatting fixed
Step 2 - Date formats fixed
Step 3 - Duplicates removed: 1 rows
Step 4 - Outliers in area_sqft: 1 removed
Step 4 - Outliers in price: 0 removed
Step 5 - Data types fixed

Final shape: (8, 8)
Cleaning complete!

```

2.9 Data Cleaning Checklist

Give this checklist to students. They should run through it at the start of every data science project.

1. `df.shape` — how many rows and columns?
2. `df.dtypes` — are all columns the correct data type?
3. `df.isnull().sum()` — which columns have missing values and how many?
4. `df.duplicated().sum()` — are there duplicate rows?
5. `df[col].unique()` — are there inconsistent text formats?
6. `df[col].describe()` — are there impossible or extreme values (outliers)?
7. Check date columns — are all dates in a consistent format?
8. After cleaning — re-run `df.isnull().sum()` to confirm zero missing values remain.

2.10 Complete Reference — Data Cleaning Problems and Fixes

Problem	How to Detect	How to Fix
Duplicate rows	<code>df.duplicated().sum()</code>	<code>df.drop_duplicates()</code>
Outliers (numerical)	<code>df.describe()</code> + IQR method	Drop rows or Winsorize (clip)
Inconsistent text	<code>df[col].unique()</code>	<code>.str.strip().str.title() + .map()</code>
Mixed date formats	<code>df[col].unique()</code>	<code>pd.to_datetime(infer_datetime_format=True)</code>
Wrong data type	<code>df.dtypes</code>	<code>.astype('category') / .astype(int)</code>
Extra whitespace	<code>df[col].unique()</code>	<code>.str.strip()</code>
Missing values	<code>df.isnull().sum()</code>	Drop / Impute / KNN (see Part 1)